CSC4760/CSC5760 Kokkos, CUDA, and MPI Homework (#3)

Anthony Skjellum, Instructor

Assigned: April 17, 2026 (Revised Never)
Due (11:59pm): April 30, 2026

Submit via Github check-in of your repo, and share repo with the "usual suspects." Make your own private repo, and store your results in a set of directories named by problem (e.g., problem0 for 0th problem, etc).

Note: (X/Ypts) – X is the points value for CSC 4760, and Y is the points value for CSC 5760.

Problem #0 (10pts/8pts):

You should do this problem first, as it will refresh your knowledge on linear and scatter mappings and help you with the next 3 problems immensely.

Imagine that you have a vector v, of length M. You have P processes, and the vector is distributed among the processes twice—first as a linear mapping, then as a scatter mapping. This exercise is about global-to-local and local-to-global mappings in either linear or scattered distributions.

Note: In later problems, you will have such vectors distributed and replicated on process topologies of shape $P \times Q$, but here we are just recalling how to map between local and global indices and processes in both mappings.

You have P processes, a vector v of length M, and youh ave the process rank value as p, with index in that process i. Find the global index I of that triple (p, i, M) with a linear inverse distribution.

Given that same global index I, the length of the array M, and the number of processes P, is it possible to find the rank of the process p, and the local index i of its scatter-mapped index? How?

Write a sequential program that takes M, P, p, and i (for the linear-mapped distribution), and returns the global index I. Then, compute and the process rank p' and local index i' that it maps to on the scatter mapping. You do not need to actually allocate or distribute an array, just do the indexing, but it may be helpful to do so. Make sure to handle cases where the array does not divide evenly into the processes. See the figure 1 for an example where $P = 4$, $M = 8$, looking at local index $i = 1$ in the linear chunk of process $p = 0$. Look at the linear mapping code provided by Dr. Skjellum for more help. Be sure to look at notes and code provided before for forward and inverse mappings of linear and scattered?

Problem #1 (15pts/12pts):

Write a program that uses `MPI_Comm_split` to create two distinct sets of sub-communicators for all processes in `MPI_COMM_WORLD`. The world size must be exactly $P \times Q$ for $P, Q \geq 1$. In the first split, group processes that have the same color when their ranks are divided by the integer Q; within these groups, perform an `MPI_Reduce` to calculate the sum of the world ranks and have the root of each sub-communicator print the result. In the second split, group processes together that have the same color when the color is computed as rank modulo Q; within these groups, have the root process perform an `MPI_Bcast` of its original world rank to all other processes in that sub-communicator. Each process should then print the value it received to demonstrate that the collective communication was isolated to processes of the same color.

Problem #2 (20pts/16pts):

Write an MPI program that builds a 2D process topology of shape $P \times Q$. On each column of processes, store a vector x of length M, distributed in a linear load-balanced fashion "vertically" (it will be replicated Q times). Start with data only in process (0,0), and scatter it down the first column. Once it is scattered on column 0, broadcast it horizontally in each process row. Allocate a vector y of length M that is replicated "horizontally" in each process row and stored also in linear load-balanced distribution; there will be P replicas, one in each process row. Using `MPI_Allreduce` or `MPI_Allgather` with the appropriate communicators, do the parallel copy $y := x$. There should be P replicas of the answer in y when you're done.

See Figure 2 below for a diagram of an example where M=15, P=4, and Q=2.

Note: Leverage your work on Problem #1 above to help do this problem.

Problem #3 (10pts/8pts):

Modify Problem #2 above, but store y in a scatter distribution (aka wrap-mapped) distribution. For this case, from global coeffient J on Q processes, then local index is $j = J/Q, q = J \mod Q$, and the number of elements per process is the same as in the linear load-balanced distribution would produce with N elements over Q partitions.

Problem #4 (20pts/12pts): CUDA Kernel

Develop a CUDA program to perform row-wise addition of a 1D vector to a 2D matrix. Initialize the host matrix A as a 3×3 grid with the following values:

$$A = \begin{bmatrix} 130 & 147 & 115 \\ 224 & 158 & 187 \\ 54 & 158 & 120 \end{bmatrix}$$

and for vector B with values:

$$B = \begin{bmatrix} 221 & 12 & 157 \end{bmatrix}$$

Transfer these to device memory and execute a parallel kernel where each thread calculates $A[row][col] + B[col]$ using flat-index mapping ($row \times cols + col$). Finally, copy the updated matrix back to the host to verify the result against the expected solution. The expected value of this updated matrix is:

$$Soln = \begin{bmatrix} 351 & 159 & 272 \\ 445 & 170 & 344 \\ 275 & 170 & 277 \end{bmatrix}$$

Problem #5 (10pts/8pts): Catastrophic Cancellation Demonstration

Using C/C++, add $1E-18$ to the a `double` variable starting with the value 1.0, 100,000 times. Output the result in high precision. Next, add the number $1E-18$ to a `double` variable starting with the value 0, also 100,000 times. Then add 1 to that sum, and compare the results in the two variables by subtracting them, and also output them in high precision. What's the difference in these two sums? (For fun, also output the results of the sums in hexadecimal to see any differences in bit patterns, so you learn how to basically inspect floating point numbers in hex for debugging.)

Problem #6 (15pts/12pts)

Using the results of Problem #1 and #3 to compute the dot product of two distinct vectors of length M, one stored initially horizontally in linear load-balanced distribution, and one stored initially vertically in linear load-balanced distribution. The scalar result should be in all processes at the end of the computation.

Hint: You can do the copy operation shown in Problems #2 and #3 to make this much easier.

CSC5760—Graduate Student Problem(s)

Problem #G1 (10pts):

1. (8pts) Modify the exercise with one stored in linear load-balanced distribution, and the other in the scatter distribution.

2. (12pts) Compute the Matrix-vector product of an $M \times N$ matrix A stored on the $P \times Q$ grid with linear load-balanced rows and columns with a vector of length N stored horizontally over the same process topology. Explain the distribution of your output vector $y := Ax$; is it in a horizontal or vertical distribution?

   (5pts–Extra credit for CSC5760) Show how to compute $z := Ay$ (where z is stored like x), and A, y are stored as above.

Problem #G2 (10pts)

Modify your Game of Life program from Homework #1, Problem #G1 to use a Kokkos parallel kernel to compute the updates at each iteration.

Hints: you may wish to change your local domain data structures to include the halos (ghost cells) as part of your local domains. This will remove the need for 'if' in the Kokkos kernel. At iteration i, your receives of halos will be into the domain to be updated. This will mean you have to have two sets of halos, one in each of the even and odd copies of your domain.

Note: CSC4760 students (aka undergrads) may do these problems for extra credit.

A Diagram of an example for problem 0

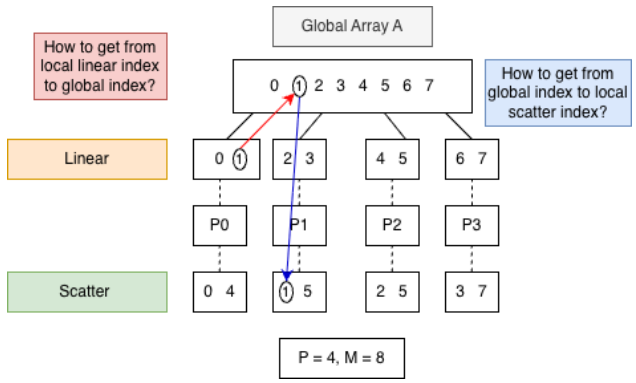

Figure 1: A diagram showing what your code should do, given an example where P=4, and M=8. Some details are purposefully omitted.

B Diagram of an example for problem 1

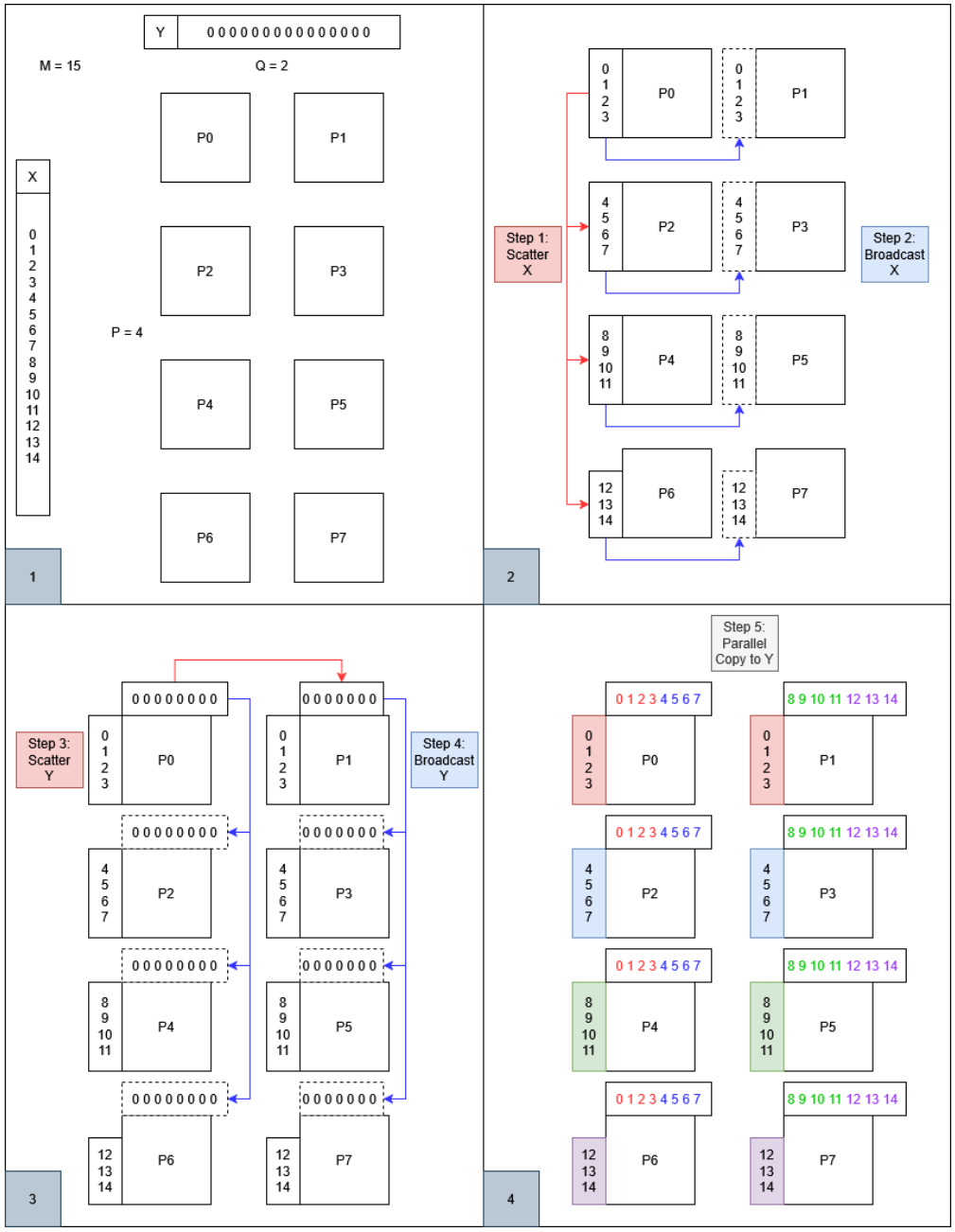

Figure 2: A diagram showing what your code should do, given an example where P=4, Q=2, and M=15. Some details are purposefully omitted.